Introducción al Machine Learning

Aprendizaje Supervisado y No Supervisado
Regresión Lineal

Licenciatura en Matemáticas Aplicadas y Computación

Facultad de Estudios Superiores Acatlán
Universidad Nacional Autónoma de México



Agenda

1. **¿Qué es la Inteligencia Artificial?** Definición y relación con el ML.
2. **¿Qué es el Machine Learning?** Aprender a partir de datos.
3. **Aprendizaje Supervisado** y **Aprendizaje No Supervisado**.
4. **Deep Learning**
5. **Regresión Lineal**
6. **Implementación en Python:** de la fórmula al código.

¿Qué es la Inteligencia Artificial (IA)?

Definición

La **Inteligencia Artificial** es la rama de la computación que busca construir **sistemas capaces de realizar tareas** que normalmente requieren **inteligencia humana**: razonar, percibir, aprender, comprender lenguaje y tomar decisiones.

La IA es un **paraguas**

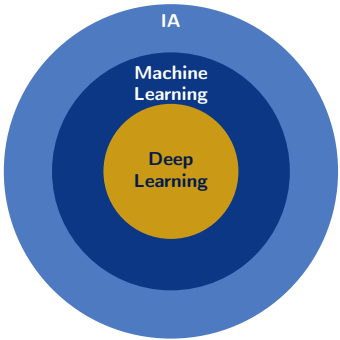
- **Sistemas expertos** (reglas lógicas).
- **Búsqueda y planificación** (algoritmos).
- **Machine Learning** (aprender de datos).
- **Deep Learning** (redes neuronales profundas).

Ejemplos

- Asistentes de voz (Siri, Alexa).
- Recomendaciones (Netflix, Spotify).
- Reconocimiento facial y de voz.
- Diagnóstico asistido por imágenes médicas.

IA, Machine Learning y Deep Learning

- **IA:** el campo amplio que busca simular la inteligencia.
- **Machine Learning:** subconjunto de la IA que **aprende de datos** sin ser programado explícitamente.
- **Deep Learning:** subconjunto del ML basado en **redes neuronales profundas**.



IA ⊃ ML ⊃ Deep Learning

¿Qué es el Machine Learning?

Definición clásica (Arthur Samuel, 1959)

"Campo de estudio que da a las computadoras la **capacidad de aprender sin ser programadas explícitamente**."

La idea

En vez de escribir **reglas** a mano, alimentamos al modelo con **datos** y él mismo **descubre los patrones** que los relacionan.

Formalización

Un programa **aprende** de la experiencia E respecto a una tarea T con medida de desempeño P, si su desempeño en T **mejora** con E.

Componentes

- **Datos:** el combustible (features, etiquetas).
- **Modelo:** la hipótesis matemática (regresión, árboles, redes...).
- **Función de pérdida:** qué tan mal predice.
- **Optimizador:** ajusta los parámetros para reducir el error.

Tipos de Aprendizaje



Supervisado: aprende con datos **etiquetados** (con "respuesta correcta").

No supervisado: busca **estructura oculta** en datos **sin etiquetas**.

Por refuerzo: un agente **interactúa** y aprende de **recompensas** y castigos.

Aprendizaje Supervisado

Idea central

El modelo aprende de **ejemplos etiquetados**: cada punto de entrada x viene acompañado de su **salida esperada** y (la "respuesta correcta").

Datos de entrenamiento

$$\{(x^{(1)}, y^{(1)}), \ldots, (x^{(m)}, y^{(m)})\}$$

- x: **features** (entrada); y: **etiqueta** (respuesta).

Dos tipos de problemas

- **Regresión:** valor **continuo** (precio, temperatura).
- **Clasificación: categoría** (spam, perro/gato).

Ejemplos

Correo *spam*, detección de fraude, diagnóstico médico.

Aprendizaje No Supervisado

Idea central

El modelo recibe datos **sin etiquetas** y debe descubrir por sí mismo **estructuras, patrones o agrupaciones** ocultas en ellos.

Clustering (agrupamiento)

- Agrupar datos **similares** en grupos (clusters).
- Ejemplo: segmentación de clientes (**K-Means**).

Reducción de dimensionalidad

- Resumir los datos en **pocas variables** conservando la información clave.
- Ejemplo: **PCA**, compresión de imágenes.

Ejemplos

Segmentación de mercado, detección de anomalías y motores de recomendación sin historial.

Supervisado vs. No Supervisado

Criterio	Supervisado	No Supervisado
Datos	Etiquetados (y conocido)	Sin etiquetas
Objetivo	Predecir la salida	Descubrir estructura
Tareas	Regresión, clasificación	Clustering, reducción
Retroalimentación	Directa (comparación con y)	Indirecta (no hay verdad)
Algoritmos	Regresión lineal, árboles, SVM	K-Means, PCA, Apriori

Regla rápida

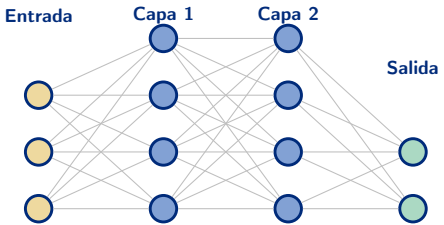
¿Tienes la respuesta correcta en los datos? $\Rightarrow$ **supervisado**. ¿Sólo tienes datos y buscas patrones? $\Rightarrow$ **no supervisado**.

Deep Learning

Idea central

El **Deep Learning** usa **redes neuronales con muchas capas** (“profundas”) para aprender representaciones cada vez más abstractas de los datos.

- **Neurona artificial:** recibe entradas, las pondera, suma y aplica una **activación** no lineal.
- **Capa oculta:** transforma la entrada en una representación intermedia.
- **Backpropagation:** el error fluye hacia atrás y **ajusta los pesos**.



¿Por qué importa?

Sin necesidad de extraer **features a mano**, las redes profundas aprenden a hacerlo solas: reconocimiento de imágenes, voz, traducción y generación de texto.

Regresión Lineal

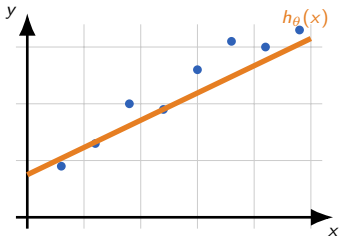
Idea central

Encontrar la **recta** que mejor se ajusta a los datos: modelar la relación entre una variable de entrada x y una salida continua y.

Hipótesis

$$h_\theta(x) = \theta_0 + \theta_1 x$$

- θ_0: **intersección** (intercept).
- θ_1: **pendiente** (peso).



Minimizamos la distancia entre los puntos y la recta.

Función de Costo: Error Cuadrático Medio (MSE)

¿Qué tan buena es la recta?

Comparamos cada **predicción** $h_\theta(x^{(i)})$ con su valor **real** $y^{(i)}$: el MSE promedia esos errores **al cuadrado**.

Definición

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)} \right)^2$$

- m: número de ejemplos de entrenamiento.
- $h_\theta(x^{(i)}) - y^{(i)}$: **error** (residuo) del ejemplo i.
- Elevar al cuadrado **penaliza los errores grandes**; el $\frac{1}{2}$ es por conveniencia algebraica.

Solución: la Ecuación Normal

Solución cerrada

Con la matriz de diseño X (una columna de 1s y las features), el θ óptimo que minimiza $J(\theta)$ se obtiene directamente:

$$\theta = (X^T X)^{-1} X^T y$$

Observaciones

- Es una fórmula **cerrada**: no itera.
- Requiere la **inversa** de $X^T X$ (cara en dimensión alta).

Implementación en Python: regresión lineal con NumPy

Dataset sintético (tamaño de casa → precio) y ajuste con la **ecuación normal**:

```python
import numpy as np

# Datos: tamano en m^2 (X) y precio en miles (y)
X = np.array([50, 60, 70, 80, 90, 100, 110, 120])
y = np.array([150, 180, 195, 230, 260, 290, 325, 355])

m = X.size
X_b = np.c_[np.ones((m, 1)), X]    # columna de unos (theta_0)

# Ecuacion normal: theta = (X^T X)^-1 X^T y
theta = np.linalg.inv(X_b.T @ X_b) @ X_b.T @ y
print("Pendiente :", round(theta[1], 2))
print("Intercepto:", round(theta[0], 2))

# Prediccion para una casa de 95 m^2
precio = theta[0] + theta[1] * 95
print("Precio estimado:", round(precio, 2), "miles")
```

Resumen de la sesión

- **IA** es el paraguas; **ML** aprende de datos; **Deep Learning** usa redes neuronales profundas.
- **Supervisado:** aprende con etiquetas (regresión, clasificación).
- **No supervisado:** descubre estructura sin etiquetas (clustering, reducción).
- **Regresión lineal:** $h_\theta(x) = \theta_0 + \theta_1 x$ minimizando el **MSE**.
- **Solución:** la **ecuación normal** $\theta = (X^T X)^{-1} X^T y$.
- En **Python**, con NumPy basta unas líneas para ajustar θ y predecir.